

Tema 6:

Apps web en dispositivos móviles

Parte I: Introducción y APIs Javascript

Tema 6: Apps web en dispositivos móviles

6.1.

Tipos de aplicaciones en móviles

(Según la plataforma de desarrollo)

Aplicaciones nativas

Desarrolladas con el SDK nativo de la plataforma y en los lenguajes soportados por ellas (*Kotlin/Java en Android, Swift en iOS*)

- Ventajas:
 - “Exprimen” al máximo el hardware
 - “*Look & Feel*” de la plataforma
- Inconvenientes
 - Nula portabilidad
 - Desarrollo costoso, especializado



Aplicaciones web

Código en **Javascript**, interfaz en **HTML/CSS**

- Ventajas:
 - Portabilidad
 - Se puede acceder a muchos APIs antes reservados a apps nativas (cámara, micrófono, agenda, geolocalización, ...)
 - Baja “barrera de entrada” para el desarrollador: “solo” conocer HTML/CSS/JS
 - Se puede escapar al control de las tiendas de apps oficiales
- Inconvenientes:
 - Rendimiento no aceptable para algunos tipos de aplicaciones (p.ej. juegos 3D)
 - No tienen el “look & feel” de la plataforma móvil
 - No se pueden vender en alguna tienda de apps oficial (App Store)
 - Algunos APIs no accesibles. Proyecto Fugu, sigue la pista de la *brecha* entre APIs nativos/APIs Web

No es una idea nueva

El motor completo de Safari está dentro del iPhone. Así que **puedes crear increíbles aplicaciones Web 2.0 y Ajax que se vean y se comporten exactamente como aplicaciones en el iPhone**. Y estas aplicaciones pueden integrarse perfectamente con los servicios del iPhone. Pueden hacer llamadas, enviar correos electrónicos y buscar una ubicación en Google Maps.

¿Y adivina qué? ¡**No necesitas ningún SDK! Tienes todo lo que necesitas si sabes cómo desarrollar aplicaciones usando los estándares web más modernos para crear aplicaciones increíbles para el iPhone** hoy mismo. Así que, desarrolladores, creemos que tenemos una gran noticia para ustedes. Pueden comenzar a construir sus aplicaciones para iPhone hoy mismo.



<https://9to5mac.com/2011/10/21/jobs-original-vision-for-the-iphone-no-third-party-native-apps/>

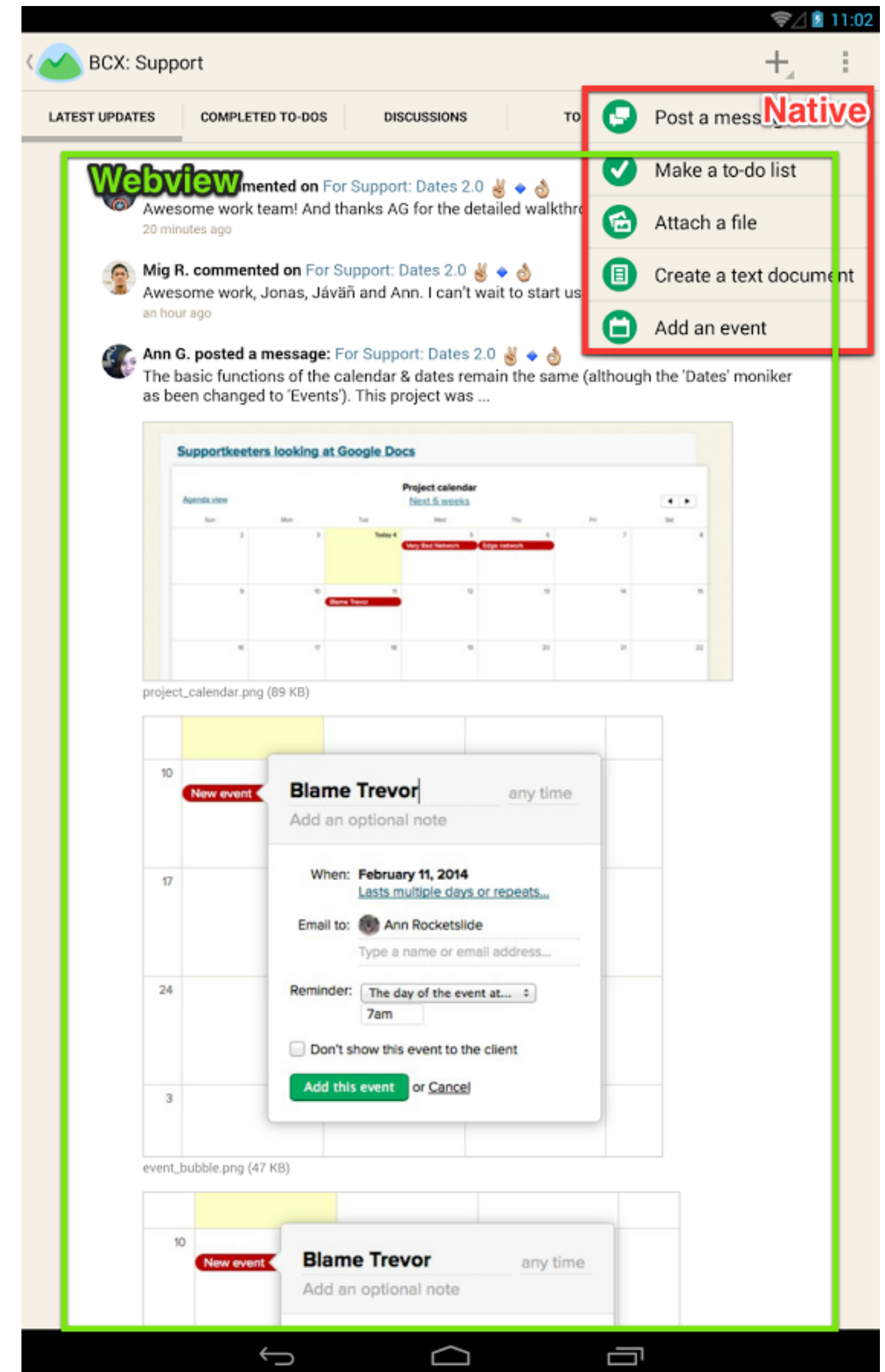
PWAs

- **Progressive Web Apps:** son aplicaciones web pero dan la impresión de ser nativas
- Se pueden instalar desde la web, cuando la visitas por primera vez
- Las veremos con más detalle la semana que viene



Aplicaciones híbridas

- La aplicación web está contenida en un “envoltorio” que es una app nativa
- Ventajas:
 - Portabilidad entre plataformas
 - Al ser “por fuera” una app clásica, se puede distribuir sin pegas en las *stores*
 - El “envoltorio” suele dar acceso a APIs nativos desde Javascript
- Inconvenientes:
 - “Look & Feel” no nativo
 - Menor rendimiento (relevante o no según el tipo de app)



Ejemplo: Capacitor

- <https://capacitorjs.com/>. La **plataforma híbrida** más conocida en la actualidad
 - Muy similar a Apache Cordova, aunque más moderno
- Dispone de un conjunto de **plugins que permiten acceder a código nativo** (Swift/ObjC/Kotlin/Java) y por tanto a los APIs nativos del dispositivo
- Podemos crear plugins propios

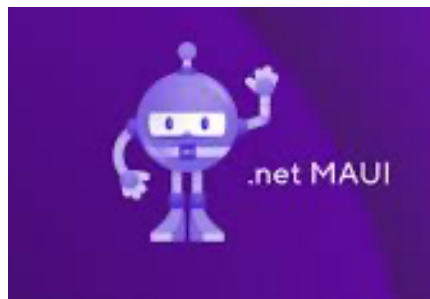


▼ Official Plugins

- Introduction
- Accessibility
- App
- Background Task
- Browser
- Camera
- Clipboard
- Console
- Device
- Filesystem
- Geolocation
- Haptics
- Keyboard
- Local Notifications
- Modals
- Motion
- Network
- Permissions
- Push Notifications
- Share
- Splash Screen
- Status Bar
- Storage
- Toast

Apps multiplataforma

- El código original se escribe en un lenguaje distinto al nativo pero luego **se compila todo a código/componentes nativos**
- Más que la portabilidad entre plataformas (que la hay), su principal ventaja es la “portabilidad” de habilidades de desarrollo. Hace **accesible el desarrollo “nativo” a desarrolladores que conocen otros lenguajes**
- Ejemplos (más info)
 - En JS: React native (React), NativeScript (Angular, Vue, Typescript, *vanilla* JS)
 - En otros lenguajes: C# - .NET MAUI, Dart - Flutter , Kotlin - Kotlin Multiplatform Mobile



Tema 6: Apps web en dispositivos móviles

6.2.

Principios generales de
diseño de apps web
móviles

La pantalla

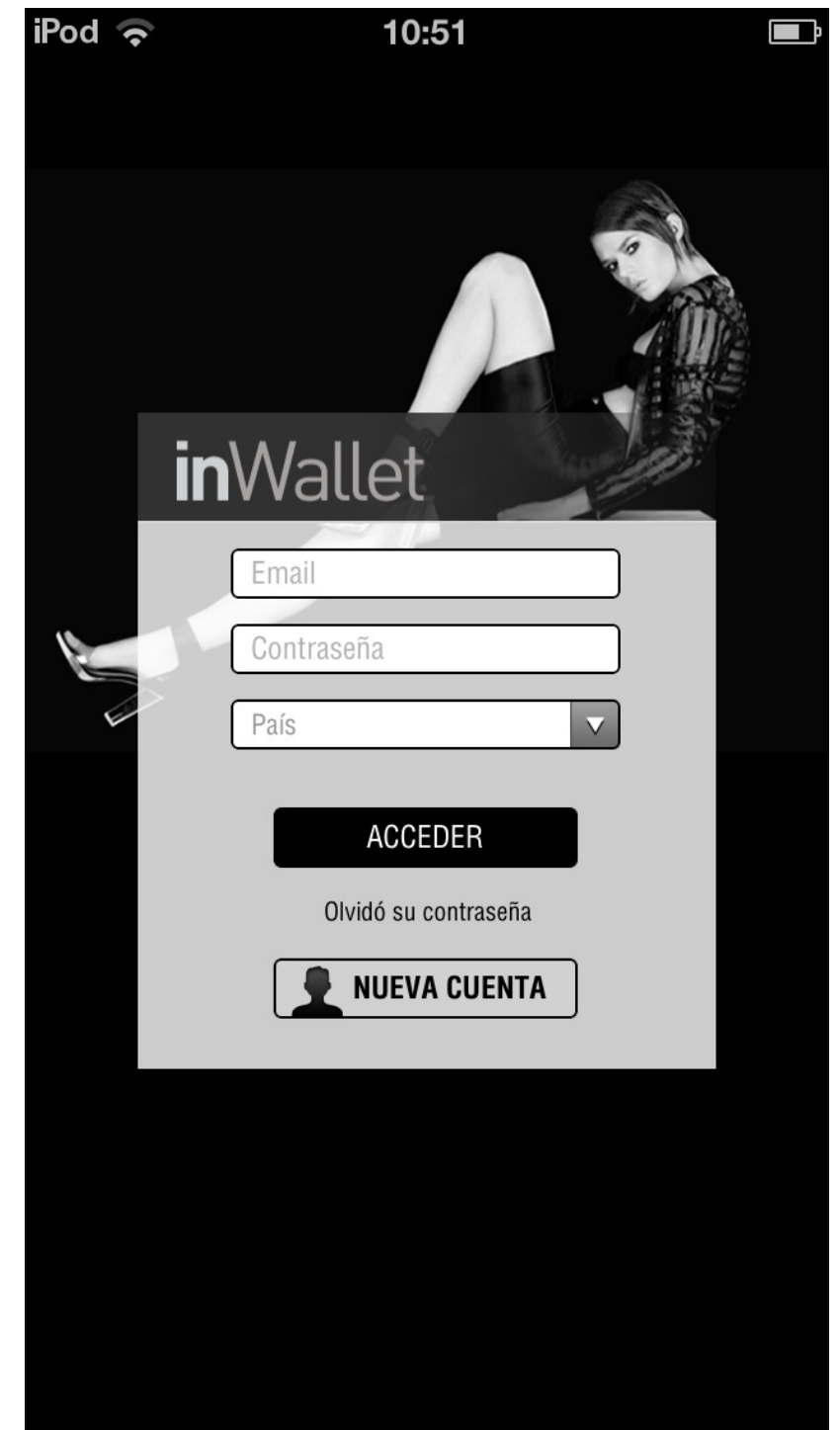
Aunque la resolución está a la par del escritorio, **no debemos colocar demasiada información** ya que la pantalla es muy pequeña

Diseño **responsive**



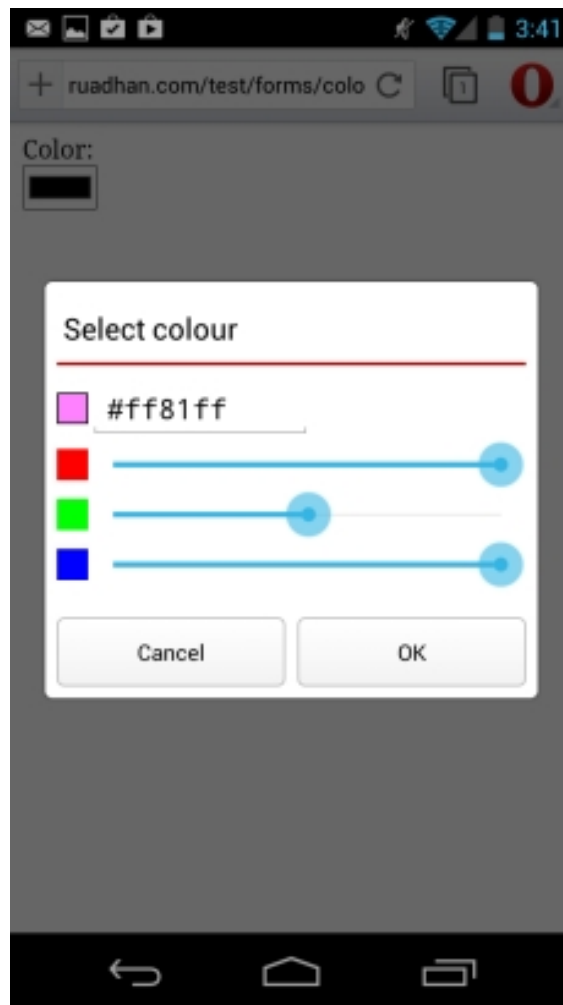
La entrada de datos es tediosa

- Introducir datos en un móvil es incómodo y tedioso.
 - La aplicación debe reducir esta necesidad al mínimo imprescindible
 - Ejemplo: recordar login y password, recordar preferencias,... (podemos usar `localStorage`)
- Los controles táctiles deben ser grandes, para poder ser pulsados cómodamente con el dedo.

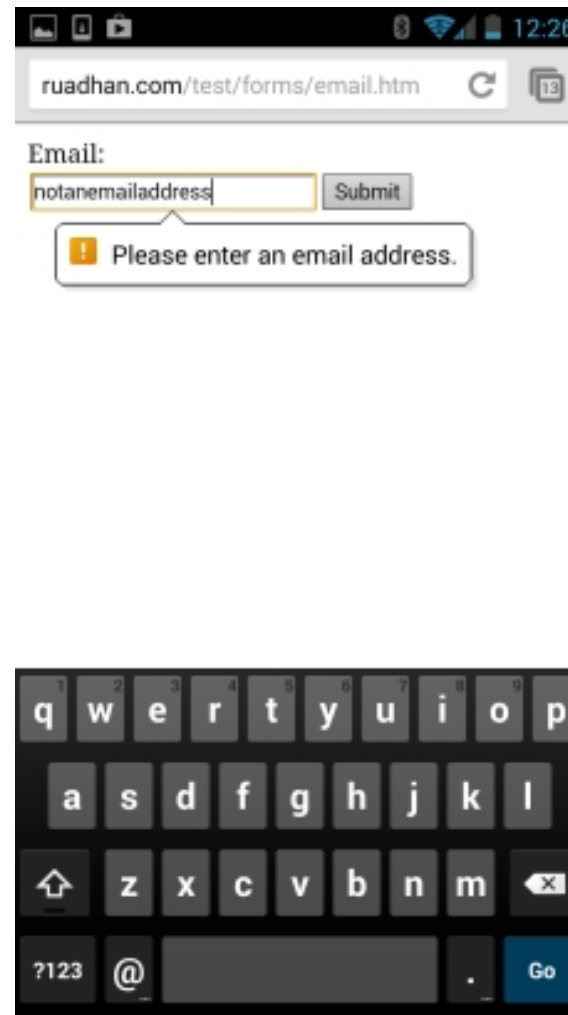


Facilitar la entrada de datos

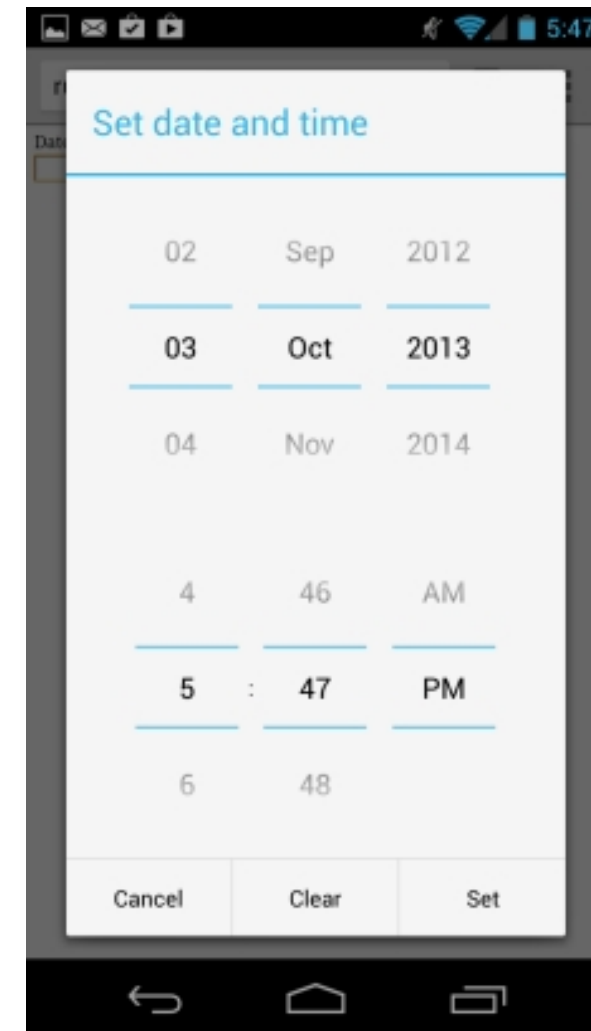
- Usar `<input>` del type más apropiado (number, date, email, color,...)



`<input type="color"/>`



`<input type="email"/>`



`<input type="datetime-local"/>`

Usar los menores recursos posibles

- Los móviles tienen restricciones de
 - Batería
 - Latencia de red
 - Memoria y capacidad de procesamiento
- Consejos habituales:
 - Reducir al máximo el número y tamaño de las dependencias



Pros of Preact

- Fast & Lightweight: Preact is just 3.5 kb in size and renders quickly, making it the best framework for building high-performing lightweight apps.
 - Compatible: It is highly compatible with React API and supports the same ECMA Script, making it efficient enough to integrate with an existing React project for enhanced performance.
- Usar rendering en el servidor (SSR, SSG) como vimos la semana pasada
 - No malgastar recursos hardware (p.ej. precisión en la localización, como luego veremos)

Code splitting

- Usar el `import()` “dinámico” de ES6

```
import { createRouter, createWebHistory } from 'vue-router';

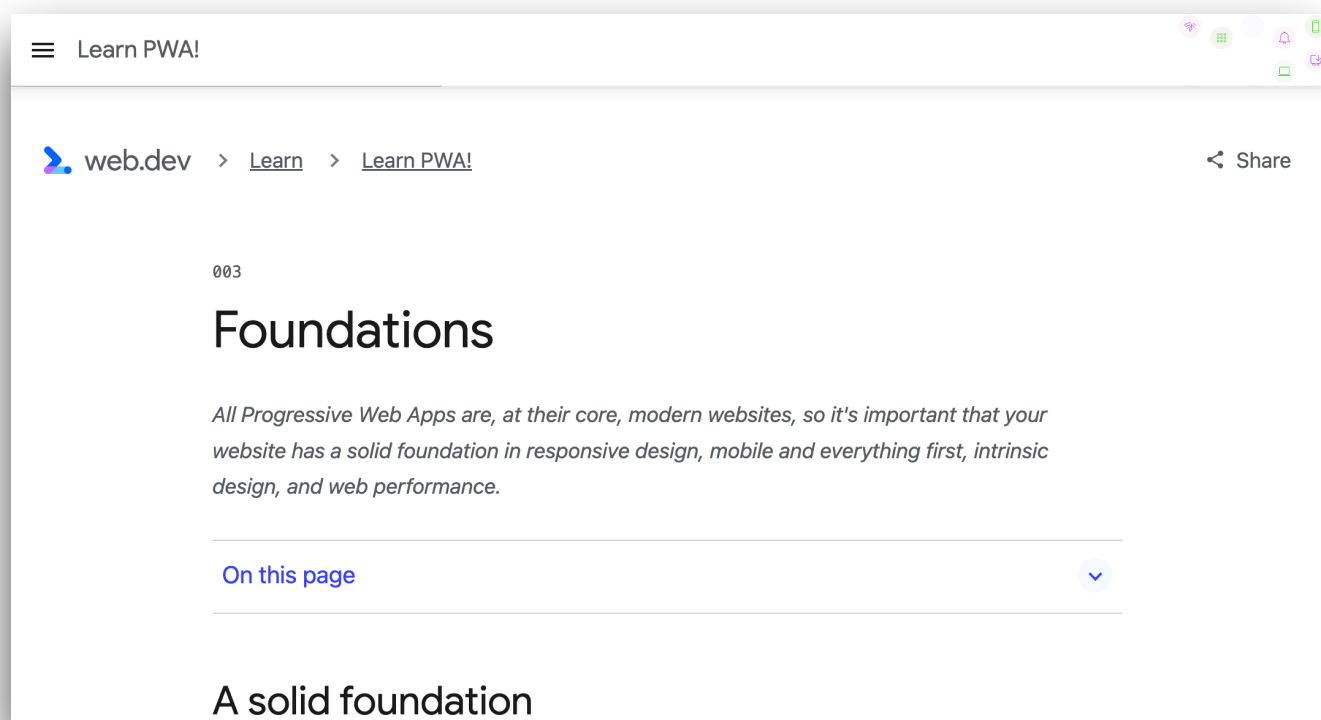
const routes = [
  {
    path: '/',
    name: 'Home',
    component: () => import('./views/Home.vue') },
  {
    path: '/about',
    name: 'About',
    component: () => import('./views/About.vue') },
];

const router = createRouter({
  history: createWebHistory(),
  routes,
});
```

- Los *bundlers* como Vite o Webpack colocan cada `import` dinámico en un archivo independiente (*chunk*) que se cargará solo cuando se necesite

Más consejos

- En web.dev, de Google: <https://web.dev/learn/pwa/foundations/>
- Charla “[Principles of mobile app design](#)”, en Google IO 2016
 - Web: [25 principles of mobile app design](#)



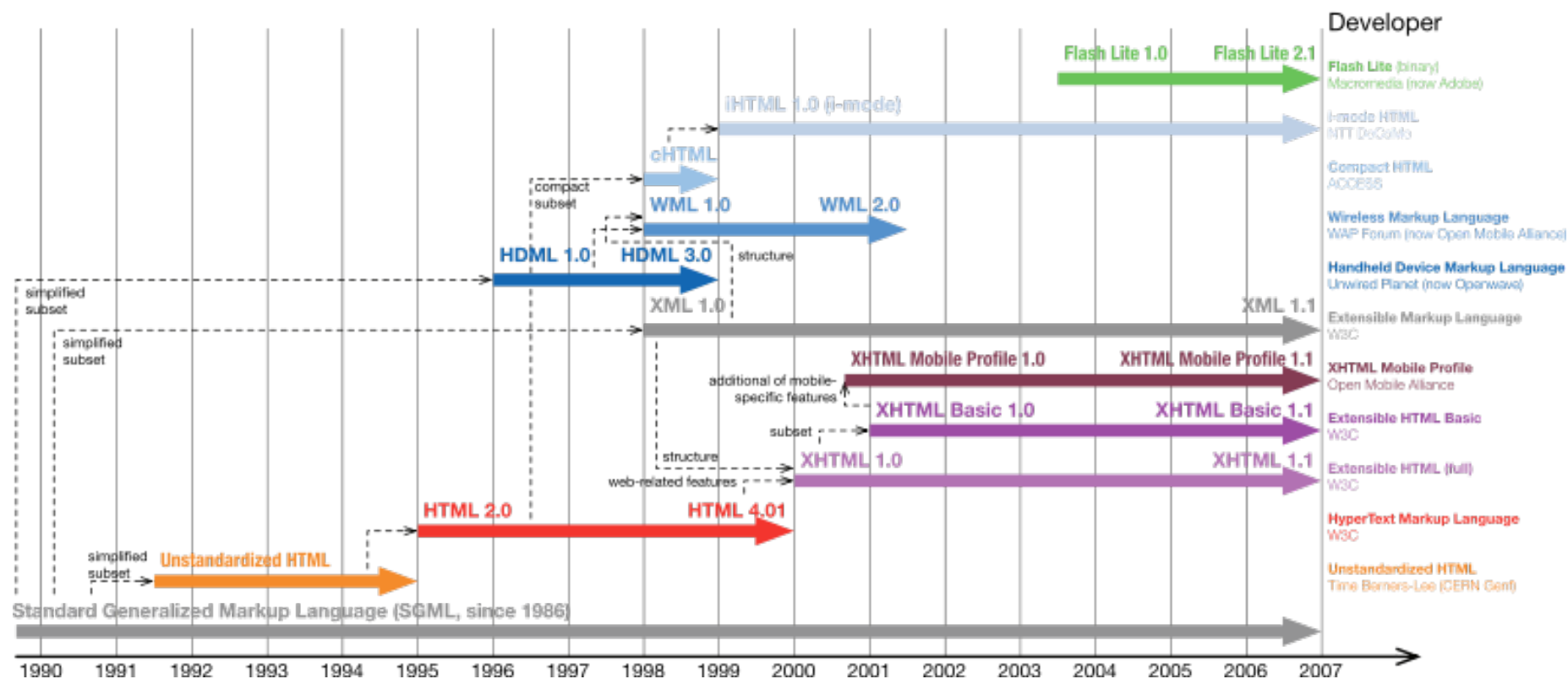
Tema 6: Apps web en dispositivos móviles

6.3.

HTML y CSS para apps
web móviles

Lenguajes de marcado

- Históricamente hay diferentes **variantes de HTML y CSS** específicas para móviles
 - XHTML Basic (subconjunto de XHTML) y CSS MP (del W3C)
 - XHTML MP (Mobile Profile, ampliación de XHTML Basic) y WAP CSS (de la OMA-Open Mobile Alliance)
- En la actualidad se suele usar **simplemente HTML5** igual que en el escritorio
https://en.wikipedia.org/wiki/XHTML_Mobile_Profile



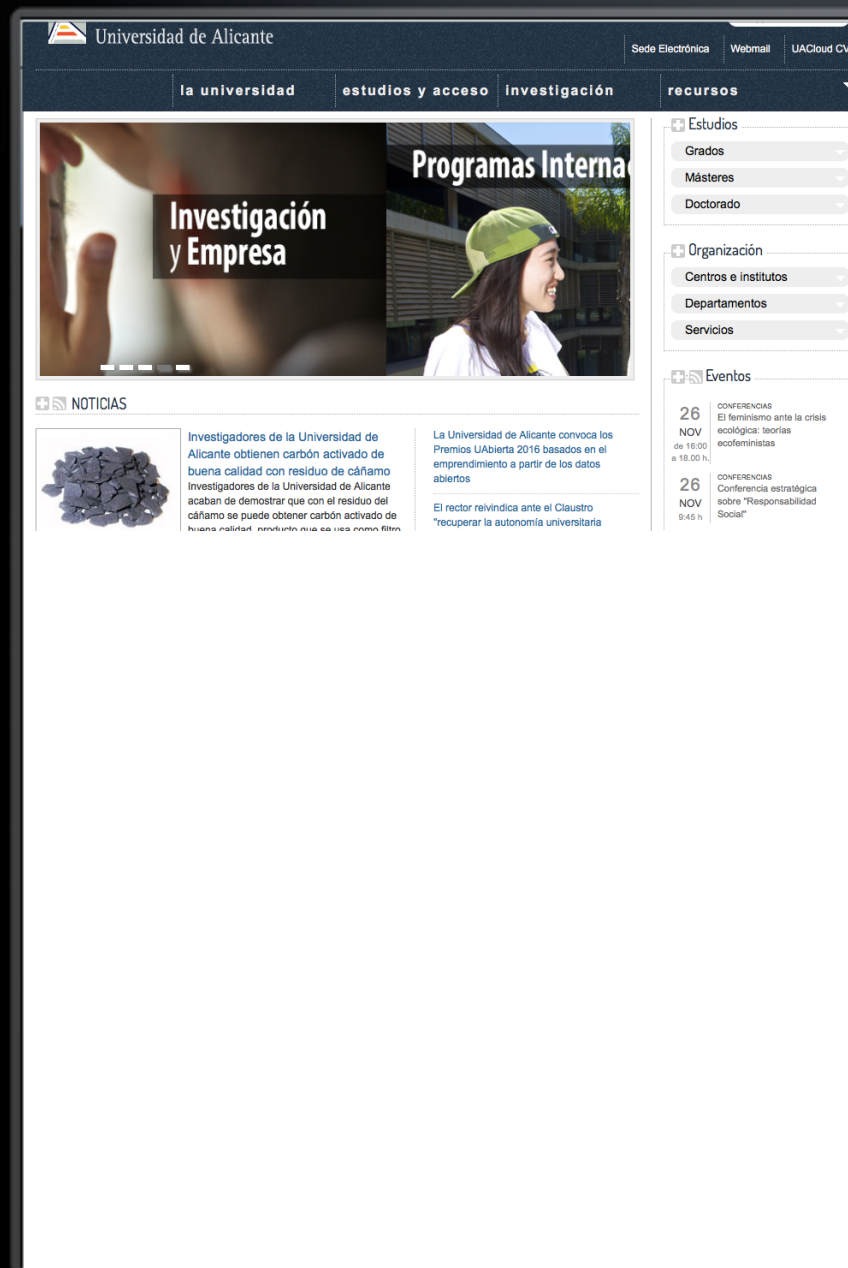
Volvamos por un momento al 2007





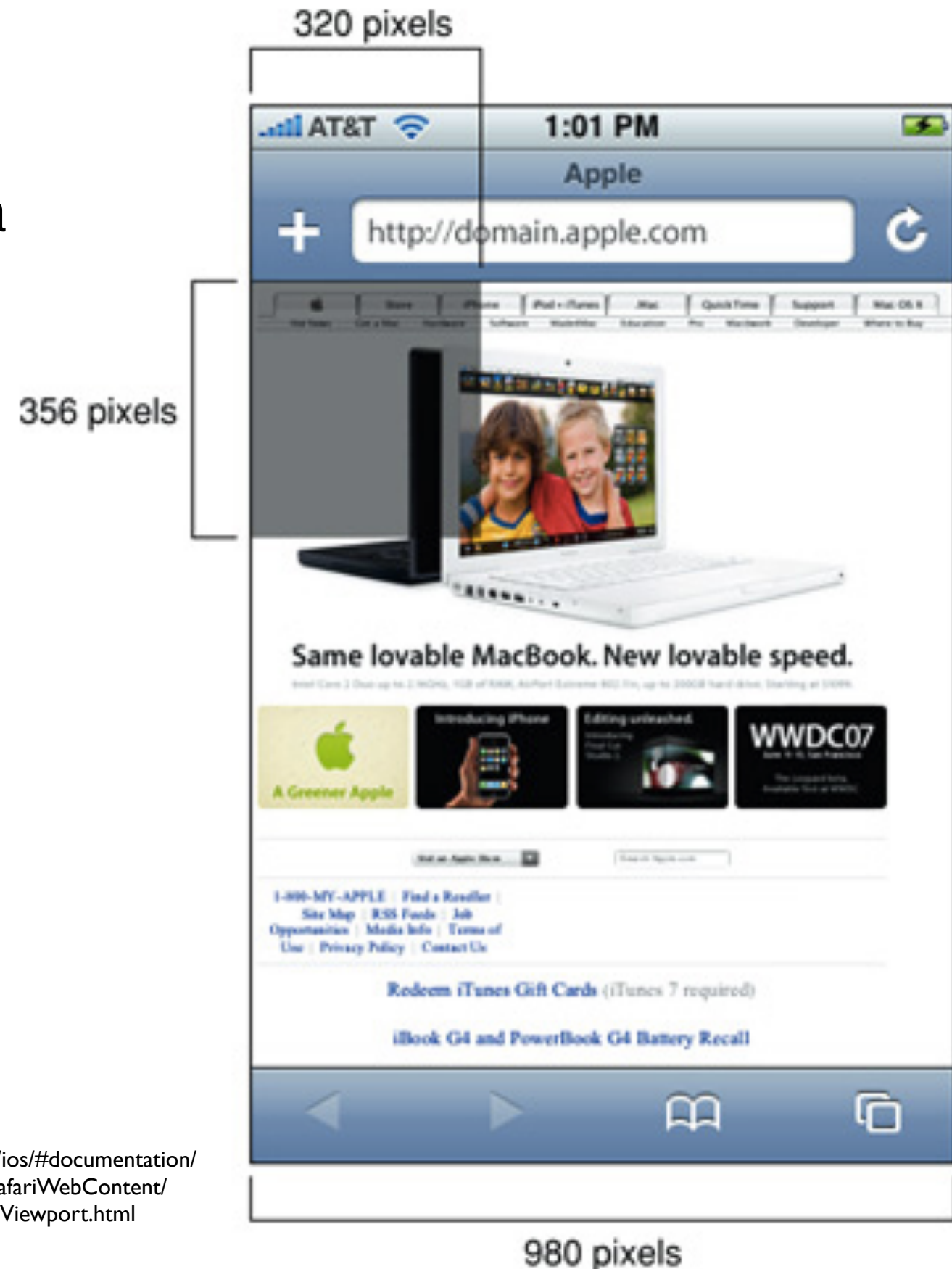
990px

viewport 980px



El *viewport*

- El dispositivo simula que tiene una resolución distinta a la real
 - El iPhone original tenía 320px de ancho, pero escalaba las páginas tomando como referencia 980px
 - Esto se hizo para que las webs aparecieran como en el escritorio (y no “cortadas” en vertical)
 - Por motivos históricos/prácticos se ha conservado la idea



<http://developer.apple.com/library/ios/#documentation/AppleApplications/Reference/SafariWebContent/UsingtheViewport/UsingtheViewport.html>

Controlar el *viewport*

- Etiqueta `<meta name="viewport">` en la cabecera
- 2 “modos de uso”:
 - Para webs con “ancho fijo”, especificar el tamaño

```
<head>  
  <meta name="viewport" content="width=640">  
</head>
```

- Para webs “fluidas”, indicar que se use el ancho del dispositivo

```
<head>  
  <meta name="viewport" content="width=device-width">  
</head>
```

Explicación más detallada: <https://web.dev/responsive-web-design-basics/>

Ejemplo sin `<meta>`

Ejemplo con `<meta>`

Un pixel no siempre es un pixel

- “Tipos” de pixels:
 - Device pixels: los físicos
 - CSS pixels: los que se usan en las reglas CSS (`screen.width`, `screen.height` reportan CSS pixels, no físicos)
- P.ej. el iPhone 16 Pro Max reporta que tiene 440 px de ancho en “portrait” (CSS pixels) cuando en realidad tiene 1320 físicos <https://yesviz.com/viewport/>
- `window.devicePixelRatio`: ¿Cuántos píxeles reales es 1 pixel CSS? (en el ejemplo anterior, 3)
- ¿Para qué se usan los píxeles “extra”?
 - Para mostrar texto más nítido (lo hace el navegador de manera “nativa”)
 - Para servir imágenes de “alta resolución” vs. “resolución estándar”: tag `<picture>`, atributo `srcset` del tag `<img>` (<https://web.dev/responsive-images/>)

Responsive web

 **UACloud**
UNIVERSIDAD DE ALICANTE



 COLOMINA PARDO, OTTO 

Ven a la app de la UA

Recuerda que puedes descargarla para **Android** o **iOS**.



Últimas



Mi alumnado
21/11/2020
20:50:50



Tutorías
21/11/2020
20:50:37



Moodle **UA**
20/11/2020
16:11:05



Solicitudes
UA
20/11/2020
13:59:11





Programa
Redes-ICE



Docencia dual



Programa
acción tutorial



Solicitudes
UA



Partes de
Mantenimiento



Pago de
recibos



Alumni**UA**





Mi alumnado
21/11/2020
20:50:50



Solicitudes
UA
20/11/2020
13:59:11

Tutorías
21/11/2020
20:50:37

Moodle **UA**
20/11/2020
16:11:05



Programa
Redes-ICE



Docencia dual



Programa
acción tutorial





Solicitudes
UA



Partes de
Mantenimiento



Pago de
recibos

Media queries

- Reglas CSS aplicables solo a ciertos tamaños de pantalla

```
<!-- vincular con una u otra hoja de estilo dependiendo de la resolución horizontal -->
<link type="text/css" rel="stylesheet" media="screen and (max-device-width:480px)"
href="smartphone.css" />
<link type="text/css" rel="stylesheet" media="screen and (min-device-width:481px)"
href="desktop.css" />

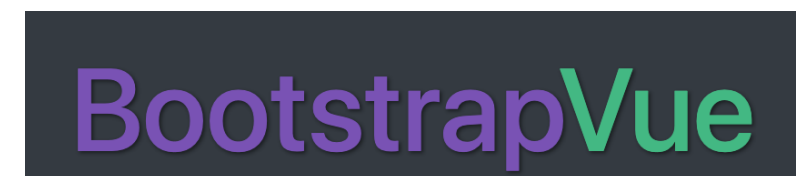
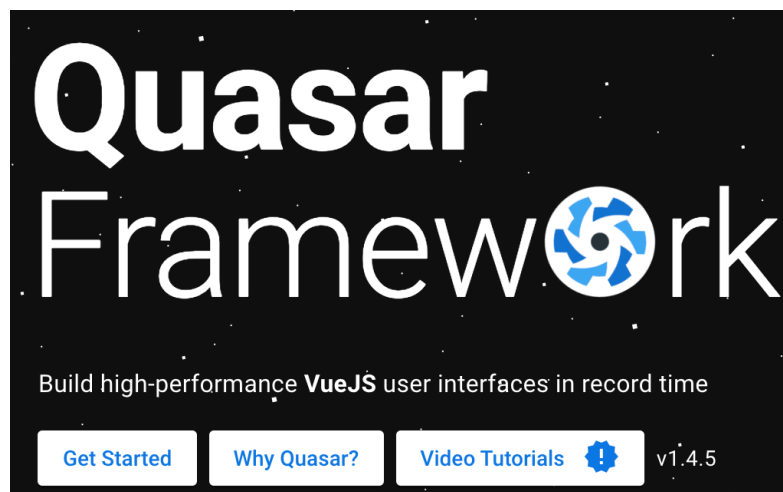
<!-- también se puede poner en el CSS "empotrado" en el HTML -->
<style>
    @media screen and (max-device-width:480px) {
        body {background-color: red;}
    }
</style>
```

Sintaxis de media queries

- **Expresiones:**
 - min- ($\geq$). p.ej. min-device-width:640px
 - max- ($\leq$). p.ej. max-device-width:640px
 - : ($=$). p.ej. device-width:640px
- **Operadores:** not, and, or, only (only screen serían dispositivos que solo soportan media="screen", típicamente móviles)
- Algunas **características** comprobables
 - device-width, device-height: medidas del dispositivo
 - width, height: medidas del viewport
 - orientation (puede ser landscape o portrait)
 - resolution (típicamente en dpi)
 - aspect-ratio, device-aspect-ratio

Componentes para Frameworks JS

- Con la popularidad de los frameworks JS al estilo React, Vue, Angular,... se han extendido las bibliotecas de componentes de UI especialmente adaptados a móvil para estos frameworks



Tema 6: Apps web en dispositivos móviles

6.4.

APIs Javascript en
móviles



Aclaración: las APIs que veremos a continuación no son exclusivas de dispositivos móviles, pero en estos es cuando podemos estar razonablemente seguros de tener el hardware necesario para que funcionen (cámara, acelerómetro, GPS,...)

Algunas APIs útiles en móviles

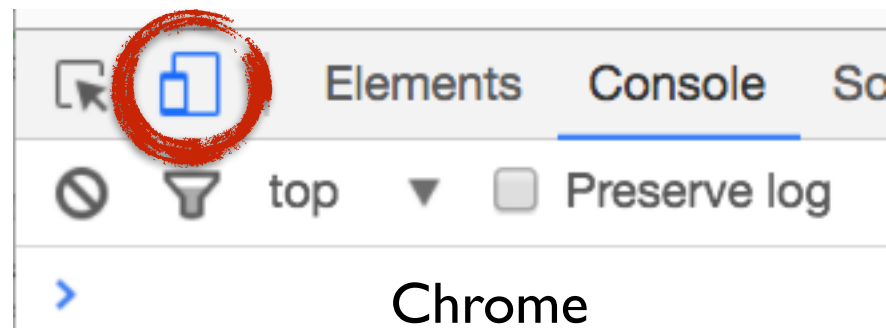
- Interacción con el hardware
 - **Cámara/Micrófono**
 - **Touch**
 - **Acelerómetro y giroscopio**
 - **Geolocalización**
 - **Battery status**
 - **Vibration**
- Uso sin conexión al servidor
 - **localStorage**
 - **Web SQL / IndexedDB:** bases de datos en el cliente
 - **service worker:** entre otras cosas, permite el funcionamiento *offline*
- Otros
 - **Payment Request API**
 - **Speech:** síntesis y reconocimiento
 - ...

<https://whatwebcando.today/>

Sitio interesante con APIs
soportadas por tu navegador
actual, documentación y demos

Para probar/depurar estas APIS

- Opción 1: en la versión de escritorio, herramientas para desarrolladores (la emulación es limitada) -



- Opción 2: en un emulador
 - <https://stackoverflow.com/questions/30901688/how-to-use-a-web-inspector-with-emulated-android-device-avd>
- Opción 3: en un dispositivo real
 - Chrome: <https://developer.chrome.com/docs/devtools/remote-debugging?hl=es-419>
 - Firefox: <https://developer.mozilla.org/en-US/docs/Tools/about:debugging>

Geolocalización

- Devuelve la **posición geográfica** del usuario (latitud, longitud)
 - Solo devuelve las **coordenadas**. Necesitaremos algún servicio adicional si queremos dibujar un **mapa** con la posición, etc. (p.ej. Google Maps o la librería JS Leaflet)
- **Métodos** de localización
 - Hay métodos con alta precisión (GPS) y baja (a partir de la dirección IP o usando la red GSM)
 - El método por el que se calcula la posición es transparente al desarrollador Javascript (no podemos elegirlo ni saberlo, solo indicar si queremos precisión alta o baja)
- **Pide permiso al usuario** cuando se ejecuta, no devuelve posición si no se concede
- Al ser una **API “antigua”** funciona con **callbacks**, no con promesas

Ejemplo en <https://stackblitz.com/edit/vitejs-vite-5zfv7w?file=index.html>

Ejemplo sencillo

- Sin chequeo de errores ni opciones de localización
- **obtener la posición:**
`navigator.geolocation.getCurrentPosition()` hay que pasarle una función *callback* (ésta recibirá la posición en un parámetro)
 - La posición recibida es un objeto con dos campos: `coords` (con la info básica: latitud, longitud, precisión, etc) y `timestamp`
 - Antes de obtener la posición es cuando se pide permiso al usuario

```
navigator.geolocation.getCurrentPosition(function(pos){  
    console.log("Estás en (" + pos.coords.latitude + "," + pos.coords.longitude + ")");  
    console.log("con precisión de " + pos.coords.accuracy + " m.");  
})
```

Gestión de errores

- Podemos pasar un segundo *callback* a `getCurrentPosition`: una función que se llamará si se ha producido algún error
 - Por ejemplo el usuario no ha dado permiso, o no hay dispositivos de localización
 - El callback de error recibe como argumento un objeto con dos campos. El más interesante es **code**, un código de error: 1:permiso denegado, 2:No se puede calcular la posición, 3:Timeout, 0>Error desconocido

```
navigator.geolocation.getCurrentPosition(mostrarPosicion, verError);

function mostrarPosicion(pos) {
    //este es el callback del ejemplo de la transparencia anterior
}

function verError(error) {
    if (error.code == 1)
        alert("No has dado permiso para ver tu posición")
}
```

Opciones de localización

- Tercer parámetro (opcional) de `getCurrentPosition`: objeto con tres campos:
 - `enableHighAccuracy` (booleano): indica si queremos una localización de precisión (p.ej. GPS) o nos basta con una aproximada (p.ej. usando la red de móvil)
 - `timeout` (nº en milisegundos) tiempo que estamos dispuestos a esperar que el dispositivo nos dé una posición. Pasado este tiempo se generará un error de `timeout`
 - `maximumAge` (nº en milisegundos) si el dispositivo tiene en cache una posición con una antigüedad inferior a esta, nos vale, no es necesario que calcule la actual.

```
//queremos alta precisión
//pero nos vale con la posición de hace un minuto
navigator.geolocation.getCurrentPosition(mostrarPosicion, verError,
                                         {enableHighAccuracy: true, maximumAge:60000});
```

Video/Audio en tiempo real

- Hay **varios APIs relativamente recientes** para obtener video/audio de la cámara del usuario, capturar imágenes estáticas, grabar el *stream*, usar todo esto para comunicación entre navegadores (*videochat*)...
- Ver los **estándares propuestos** en el grupo de interés del W3C “Web Real-Time Communications” (algunas demos)
- De menor a mayor complejidad
 - HTML media capture
 - Media capture (`getUserMedia`)
 - `MediaRecorder`

HTML media capture

- Permite **seleccionar imagen/video/audio** o **capturarlo** de la cámara/micrófono usando un `<input type="file">`

```
<!-- con "capture" se captura en ese momento, si no, se toma de la galería -->  
<input type="file" accept="image/*" capture>  
<input type="file" accept="video/*">  
<input type="file" accept="audio/*">
```

- Combinando esto con Javascript podemos:
 - Mostrar la imagen en la página en un tag `<img>` (*ejemplo*)
 - Manipularla usando el API Canvas (API de dibujo 2D)
 - Subir la imagen al servidor con `fetch`
 - ...



¡¡Esto no funciona en el escritorio, solo en navegador móvil!!

Media Capture and Streams: conceptos previos

- MediaStream stream con audio y/o vídeo, tiene 1 o más *tracks*
 - Podemos asignarlo a una etiqueta `<video>/<audio>` con la propiedad `srcObject`
- MediaStreamTrack 1 única pista de audio o vídeo
- Blob: datos binarios, usado para archivos, imágenes,...
 - `URL.createObjectURL(blobInstance)`: pasar de Blob a URL para `<img src="">`
- Canvas: área rectangular para dibujar en pantalla
 - Context: el “sitio” donde dibujamos, objeto que implementa todos los métodos para dibujar

Media Capture and Streams

(a.k.a. getUserMedia)

```
<video id="cam_video">Aquí aparecerá el video</video>
```

```
async function getStreamAndPlay() {  
  var stream = await navigator.mediaDevices.getUserMedia({video: true})  
  var video = document.getElementById('cam_video');  
  video.srcObject = stream;  
  video.play()  
}
```

<https://codepen.io/ottocol/pen/yPWoWP?editors=1010>

En el objeto pasado a **getUserMedia** se puede pedir si queremos solo video, también audio, qué resoluciones preferimos y qué cámara si hay más de una, por ejemplo

```
{  
  audio: true,  
  video: {  
    width: { min: 1280 },  
    height: { min: 720 }  
  }  
}
```

(más info en <https://developer.mozilla.org/es/docs/Web/API/MediaDevices/getUserMedia>)

¡Primero chequear el soporte!

- Chequear que el navegador actual soporta el API
- Típicamente comprobar que el objeto/método que queremos usar no es **undefined**

```
if (navigator?.mediaDevices?.getUserMedia) {  
    //soporta el API  
    ...  
}
```

Capturar fotos

- ImageCapture (Android desde hace años, iOS desde marzo 2025)

```
<video id="video" style="height: 180px; width: 240px;"></video>
<button id="boton">Tomar Foto</button>
<img id="imagen" width="240" height="180">
```

```
var stream

document.addEventListener("DOMContentLoaded", async function(){
  stream = await navigator.mediaDevices.getUserMedia({video:true})
  var video = document.getElementById("video")
  video.srcObject = stream
  video.play()
})

document.getElementById("boton").addEventListener("click", async function(){
  var capt = new ImageCapture(stream.getVideoTracks()[0])
  var foto = await capt.takePhoto()
  var img = document.getElementById("imagen")
  img.src = URL.createObjectURL(foto)
})
```

Capturar fotos “legacy”

```
<video id="player" controls autoplay></video>
<button id="capture">Capturar</button>
<canvas id="snapshot" width=320 height=240></canvas>
```

```
var player = document.getElementById('player');
var snapshotCanvas = document.getElementById('snapshot');
var captureButton = document.getElementById('capture');

captureButton.addEventListener('click', function() {
    var context = snapshotCanvas.getContext('2d');
    // Pintar el frame del video al canvas .
    context.drawImage(player, 0, 0, snapshotCanvas.width, snapshotCanvas.height);
});

var handleSuccess = function(stream) {
    player.srcObject = stream;
};

navigator.mediaDevices.getUserMedia({video: true}).then(handleSuccess);
```

De <https://developers.google.com/web/fundamentals/media/capturing-images/?hl=es>
Ver demo en <https://codepen.io/ottocol/pen/zYYQXWq?editors=1010>

Capturar streams

- MediaStream Recording API: MediaRecorder

```
<video id="cam_video"></video>
```

```
let stream = await navigator.mediaDevices.getUserMedia({video: true})

var recorder = new MediaRecorder(stream);
recorder.addEventListener('dataavailable', function(e) {
  video = document.getElementById("cam_video")
  video.src = URL.createObjectURL(e.data);
  video.loop = true
  video.play();
});
recorder.start();
setTimeout(function(){
  recorder.stop();
  console.log("Grabado!!")
}, 2000);
```

Blob

Obtenemos el stream de video de la cámara, ponemos un MediaRecorder a grabar con **"start"** y 2 segundos después paramos la grabación (**stop**) y para verla de forma sencilla, la convertimos de Blob a URL y se la ponemos a un <video> HTML ya en la página

<https://codepen.io/ottocol/pen/RxwoNB?editors=1010>

Más ejemplos en <https://developers.google.com/web/fundamentals/media/recording-video?hl=es>

Detectores de formas

- **Caras, textos, códigos de barras**
- Estándar en “incubación”, Android/últimas versiones de iOS

```
<video id="video"></video>
```

```
const video = document.getElementById('video')
const faceDetector = new FaceDetector()
let caras = faceDetector.detect(video)
console.log("Num. caras detectadas: " + caras.length)
caras.forEach(cara => {
  cara.landmarks.forEach(landmark => {
    if (landmark.type == 'eye') {
      console.log("Ojo en " + landmark.locations[0].x + "," +
                  landmark.locations[0].y)
    }
  })
})
});
```

Hay que activarlo en el navegador

En Chrome *desktop* hay que activar un flag del navegador (Experimental Web Platform Features) [Ver instrucciones](#). En Safari Desarrollo > Conmutación de funciones > buscar y marcar “shape detection”

Demo: <https://codepen.io/ottocol/pen/OOexmR>

Demo: <https://whatpwacando.today/face-detection/>

Demo de códigos de barras: <https://codepen.io/ottocol/pen/WNEBjjj?editors=1111>

Buen artículo de introducción al API (ejemplo base de la demo anterior) <https://blog.arnellebalane.com/introduction-to-the-shape-detection-api-e07425396861>

Tutorial de web.dev: <https://web.dev/shape-detection/>

Almacenamiento offline

- Interesante porque a veces puede **fallar la conectividad**
- **Almacenamiento local:** bases de datos en el cliente
 - localStorage (ya lo vimos, sencillo de usar pero no permite hacer búsquedas como en una BD)
 - IndexedDB (iOS/Android)
- **Cache** de recursos de red (HTML, CSS, JS) para que la aplicación se pueda seguir usando en la medida de lo posible
 - cache manifest (*“deprecated”*)
 - service workers (*lo veremos la semana que viene*)

IndexedDB

- API para trabajar con bases de datos de un tipo “peculiar”
- Almacena objetos (sin esquema fijo)
- Cada objeto debe tener una **clave**
- **Indices**
 - Se pueden crear para uno o varios campos
 - Sirven para ordenar/hacer búsquedas simples
 - Mismo valor
 - Valor en un rango
- **Cursores** para iterar sobre los datos

Es como un almacén de objetos clave/valor con búsquedas por índice

Wrappers de IndexedDB

- IndexedDB es de “demasiado bajo nivel” para tareas complejas
- Wrappers: capas de abstracción sobre la API
 - Facilitan su uso
 - Tipicamente sincronizan la BD local con una en la nube
- Ejemplos
 - **Dexie:**
 - **PouchDB:** ejemplo TO-DO: <https://codepen.io/astol/pen/qbbOXE?editors=1010>

```
const collection = db.friends
  .orderBy('age')
  .filter((friend) => /pepe/i.test(friend.name));

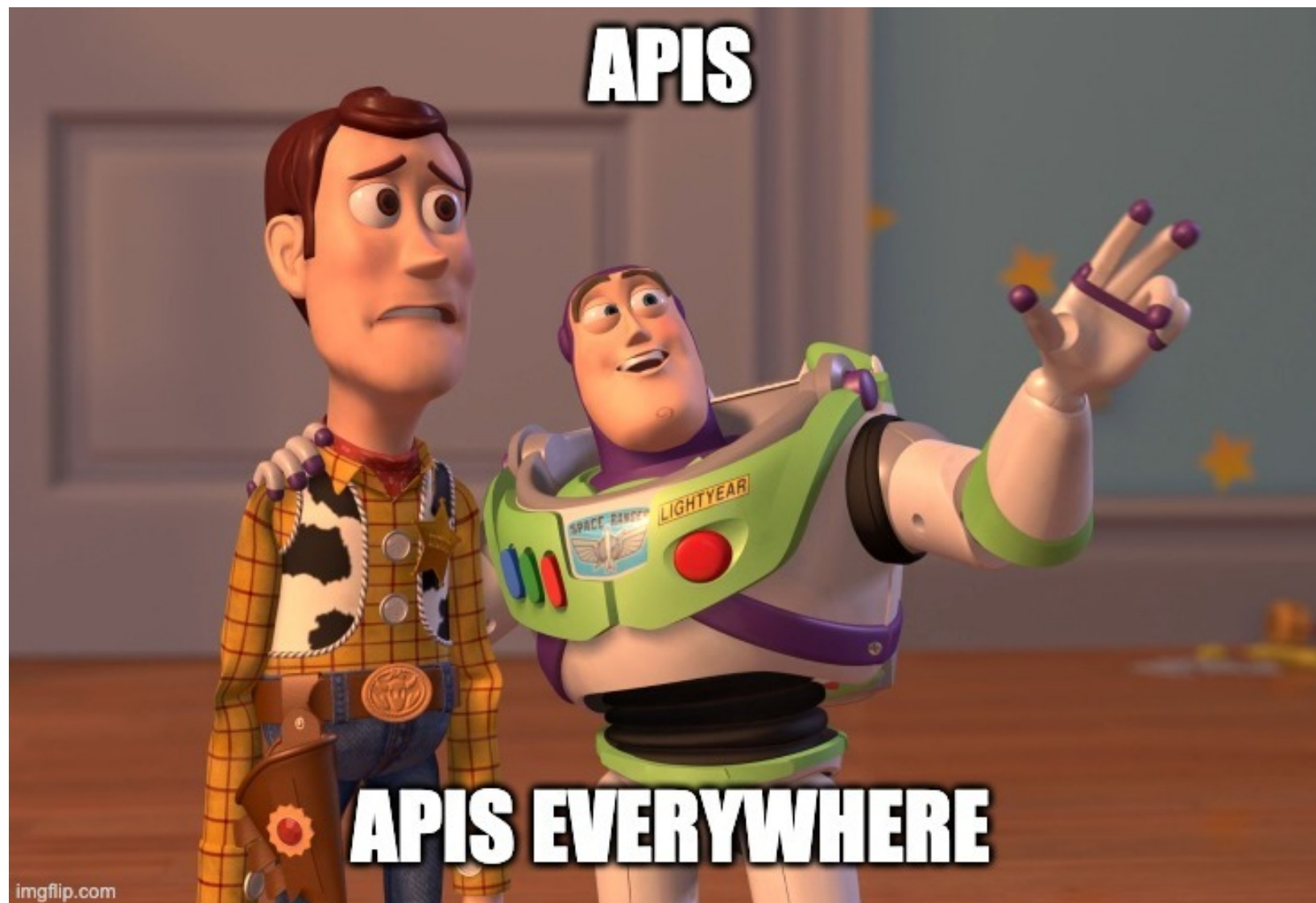
const result = await
collection.offset(50).limit(25).toArray();
```

Query en Dexie

```
db.find({
  selector: {
    name: {$gt: 'joy'}
  },
  sort: ['name'],
  limit: 10
});
```

Query en PouchDB

Otras APIs...



Pointer events

- Unifican **ratón, touch, stylus**
- **Eventos** `pointerdown`, `pointermove`, `pointerup`
 - Equivalentes a los correspondientes de ratón: *mousedown* (pulsado botón), *mousemove*, *mouseup* (soltado botón)
 - Valen para ratón y táctil (con el dedo, pen,...)
 - Existen también touch events, solo para touch
- **Propiedades** del evento:
 - Mismas que `mouseevent` más algunas propias de táctil como *width*, *height* (tamaño del área de contacto, *pressure* (entre 0 y 1))
- No implementa “gestos” (*swipe*, *pinch-to-zoom*, ...), pero sí hay muchas librerías que lo hacen, por ejemplo [hammer.js](#)

Orientación de la pantalla

- Si no necesitamos posición 3D sino solo detectar cambios de orientación entre modo **vertical** (*portrait*) y **horizontal** (*landscape*)
- **screen.orientation**: <https://developer.mozilla.org/en-US/docs/Web/API/Screen/orientation>
- Detectar cambios: evento **orientationchange** del objeto **window**

```
window.addEventListener('orientationchange', function(){  
    //solo puede ser 0,90,180,270  
    console.log("Angulo actual: " + screen.orientation.angle)  
    console.log("Tipo de orientacion: " + screen.orientation.type)  
})
```


Orientación en 3D

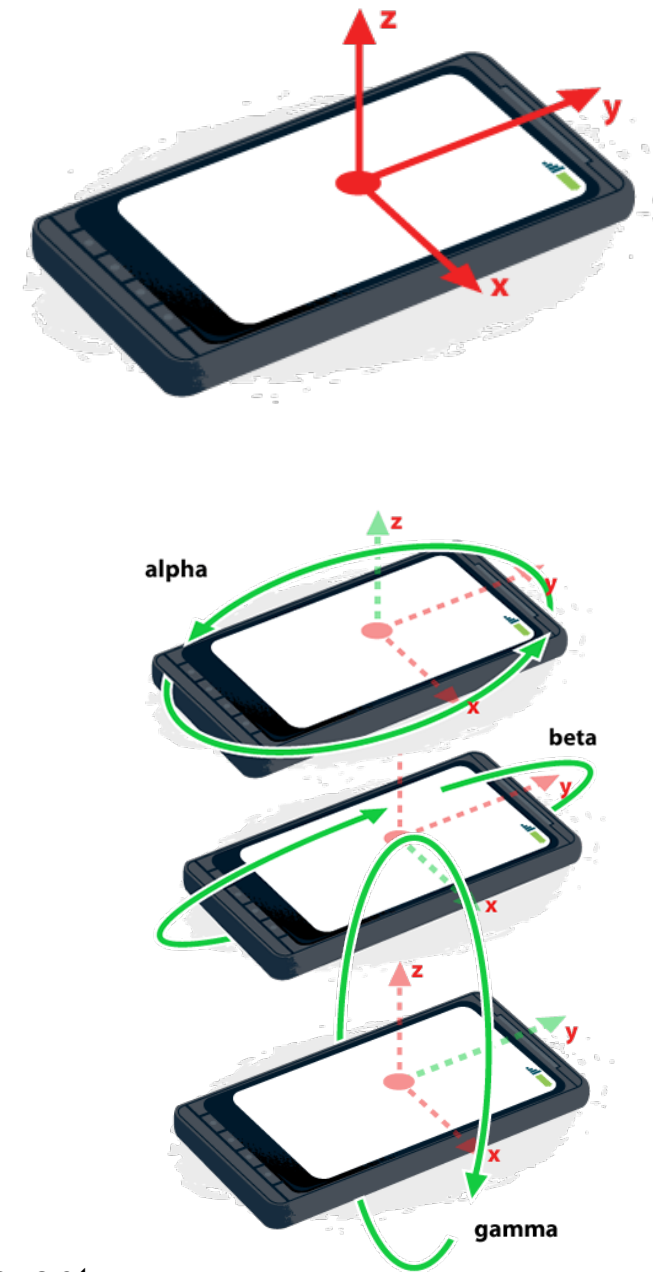
- Con el **acelerómetro** y/o **giroscopio** y **magnetómetro** se puede detectar el movimiento (evento 'devicemotion') y la posición actual en 3D (evento 'deviceorientation')
- Los eventos se disparan x veces por segundo (típicamente 50-60)
- `deviceorientation` informa de ángulos actuales con respecto a los ejes 3D (alpha-z, beta-x, gamma-y)
- `devicemotion` informa de aceleración y velocidad de rotación

Explicación básica y demo:

`deviceOrientation`: <https://progressier.com/pwa-capabilities/device-orientation-event>

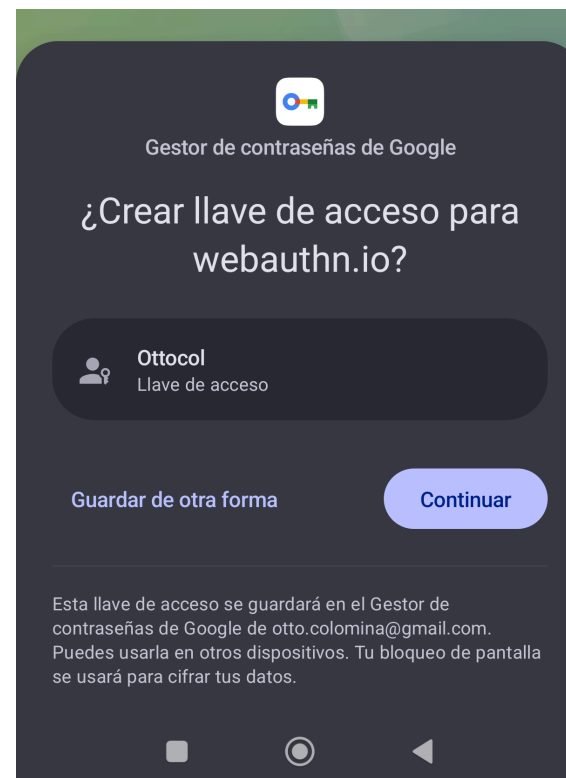
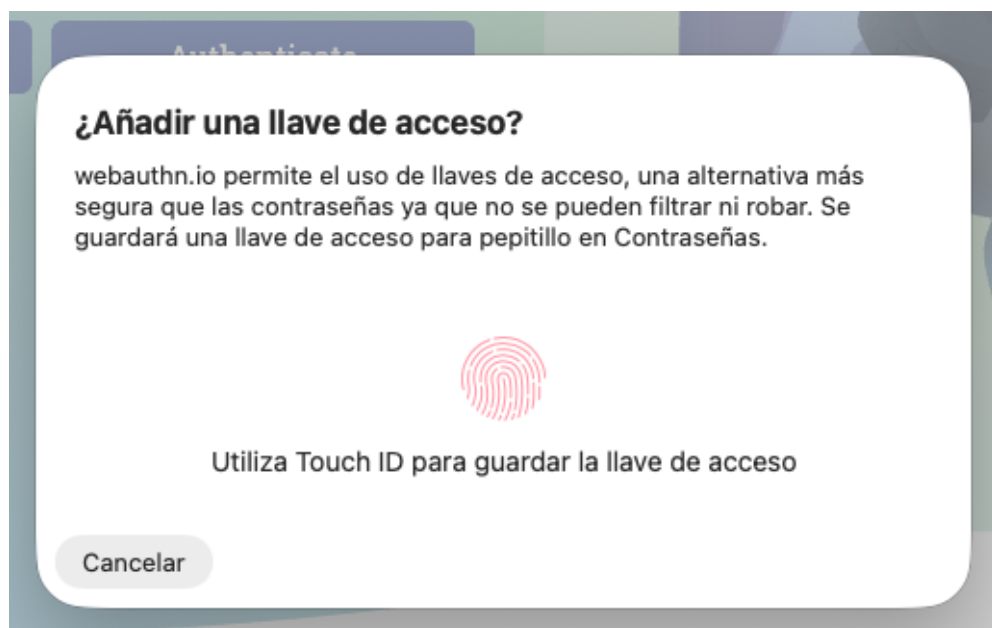
`deviceMotion`: <https://progressier.com/pwa-capabilities/device-motion-event>

En Android, alfa se mide con respecto al norte, en iOS es arbitraria y además para usar la API hay que pedir permisos al usuario



WebAuthn

- Autenticación con **passkeys** (“llaves de acceso”): par de claves de criptografía asimétrica (clave pública/privada)
 - La pública se guarda en el servidor de la app en la que queremos autenticarnos
 - La privada se guarda en nuestro dispositivo
- La autenticación biométrica se usa para poder desbloquear la clave privada almacenada en el dispositivo



La clave privada de la **passkey**:

- Es única por servicio (no se reutiliza entre webs/apps)
- Está restringida al dominio → una app no puede usar la passkey de otra,
- Está dentro del enclave seguro del dispositivo
- No se puede exportar,

WebAuthn: proceso

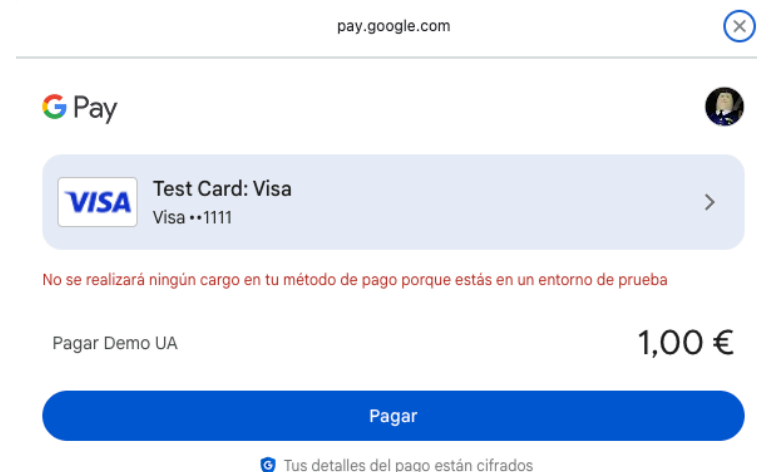
- Creación de la *passkey*
 - El servidor envía cierta información en JSON (*challenge*, id usuario, ...)
 - El dispositivo del usuario le pide autenticación biométrica (*cara, huella... lo decide el SO*)
 - El dispositivo genera la *passkey* y envía la clave pública al servidor
- Autenticación con la *passkey*:
 - El servidor envía cierta información en JSON (muy similar a la que se usó para crear la *passkey*)
 - La autenticación biométrica desbloquea la clave privada en el dispositivo
 - El dispositivo firma la información con su clave privada y la envía al servidor
 - El servidor valida la firma

Demo: <https://webauthn.io/>

Tutorial: <https://web.dev/articles/passkey-registration?hl=es-419>

Payment Request

- Objetivo: simplificar la **introducción de los datos de pago** del usuario para hacer una compra online
 - No contacta con el banco, ni con VISA/Mastercard... solo genera un token que se puede enviar a nuestro servidor para que a través de una pasarela de pago (p.ej. Stripe) lo lleve a cabo
 - En Chrome se puede “conectar” con los métodos registrados en el Google Pay del dispositivo
 - Apple tiene un equivalente para Apple Pay: Apple Pay on the web



Demo simple online: <https://output.jsbin.com/panivog/>

Fuente: <https://jsbin.com/panivog/edit?html,js>

Tutorial: <https://developers.google.com/web/fundamentals/payments/?hl=es>

Web Share

- Como el compartir  de las apps nativas. Se puede compartir una URL, texto, o conjunto de File

```
const shareData = {
  title: 'UA',
  text: 'Ven a estudiar a la UA!',
  url: 'http://www.ua.es'
}

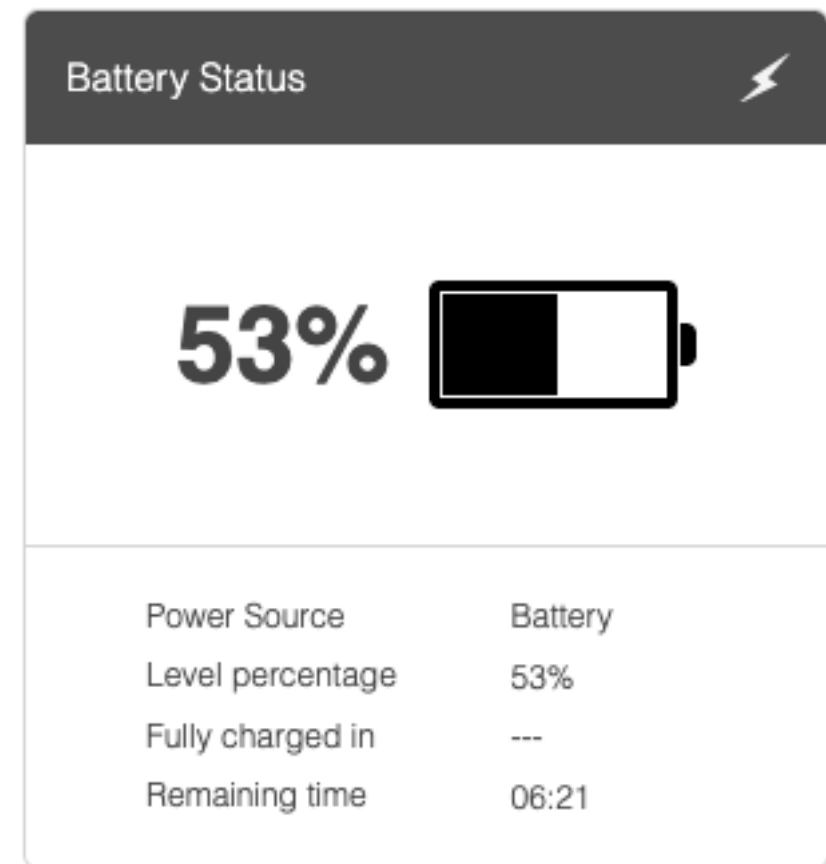
// El share debe ser disparado por un evento generado por el usuario
document.getElementById("boton").addEventListener('click', async () => {
  try {
    await navigator.share(shareData)
    console.log('OK')
  } catch(err) {
    console.log('Error: ' + err)
  }
});
```

Battery Status

- `navigator.getBattery()` devuelve una promesa que cuando se resuelve nos da los datos en JSON

Demo online:

<https://pazguille.github.io/demo-battery-api/>



No soportado en iOS por motivos de privacidad según Apple (para evitar *fingerprinting*, que los servidores puedan identificar y reconocer el dispositivo del usuario) <https://www.zdnet.com/article/apple-declined-to-implement-16-web-apis-in-safari-due-to-privacy-concerns/>

Interacción por voz

- **Speech Synthesis:** soporte en iOS y Android [[demo](#)]

```
let utterance = new SpeechSynthesisUtterance("Yo no soy ni voy a ser tu bizcochito");  
speechSynthesis.speak(utterance);
```

- **Speech Recognition:**
 - Soporte en Android e iOS pero no del estándar completo (no como el de la síntesis)
 - El reconocimiento se hace **online**, no es local [[demo](#)]

Algunas referencias

- **Web dev**: consejos de diseño, info sobre APIs en escritorio y móvil <https://web.dev/learn/>
- **Mozilla Development Network**: documentación, artículos, tutoriales (<https://developer.mozilla.org/en-US/docs/Web/API>)
- **Centradas en iOS** (casi todas las demás están centradas en Android)
 - El **blog de webkit**, el motor del navegador Safari <https://webkit.org/blog>
 - El sitio de **Maximiliano Firtman** artículos, charlas, etc sobre apps móviles web <https://firt.dev/learn/> (interesante aunque lo ha dejado de actualizar)